



LAPORAN PRAKTIKUM

Tugas 1: RESTful API Murni dengan Express.js

Mata Kuliah Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Fellisa Anastasya

NIM : 2428240094

Kelas : SI5B

Nomor : 26

Topik : Toko Gadget (Smartphone)

**PROGRAM STUDI SISTEM INFORMASI UNIVERSITAS
MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027**

Identitas Praktikum

Nama Mahasiswa	Fellisa Anastasya
NPM	2428240094
Kelas	SI5B
Pertemuan Ke-	6
Topik / Judul Praktikum	Tugas 1: RESTful API Murni dengan Express.js - Toko Gadget: Smartphone (Topik 26)
Tanggal Pelaksanaan	4 Oktober 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Membangun RESTful API murni dengan Express.js untuk resource smartphone (Topik 26) sesuai konvensi REST: nama resource jamak huruf kecil berpisah tanda hubung, tanpa prefix, dan tanpa kata kerja. API terdiri dari enam endpoint, yaitu GET /smartphones, GET /smartphones/:id, POST /smartphones, PUT /smartphones/:id, DELETE /smartphones/:id, dan filter GET /smartphones?merek=
2. Menerapkan penggunaan method HTTP sesuai fungsinya, yaitu GET untuk membaca data, POST untuk menambah data baru, PUT untuk mengganti seluruh isi data, dan DELETE untuk menghapus data. Seluruh field wajib topik 26 (merek, model, ramGb, penyimpananGb, dan harga) divalidasi dengan status 400, sedangkan data atau endpoint yang tidak ditemukan ditangani dengan status 404.
3. Menghasilkan response JSON murni pada seluruh endpoint. Response GET mengembalikan data langsung tanpa pembungkus, sedangkan response

POST, PUT, DELETE, dan error memakai struktur { status, message, data } dengan status code yang tepat (200, 201, 400, 404), serta tidak menggunakan `res.send()` berisi teks atau HTML.

II. Dasar Teori

REST (Representational State Transfer) adalah gaya arsitektur untuk merancang layanan web di atas protokol HTTP. Pada REST, sebuah resource diidentifikasi melalui URL dan diakses menggunakan method HTTP. Resource adalah konsep bisnis yang direpresentasikan, bukan proses dari aplikasi tersebut.

Resource pada praktikum ini adalah data smartphone, sehingga URL yang digunakan adalah `/smartphones`. Konvensi penamaan URL REST memakai kata benda jamak, huruf kecil, dipisahkan tanda hubung, tanpa prefix, dan tanpa kata kerja. Penulisan seperti `/getSmartphones` atau `/deleteSmartphone/1` karena itu tidak sesuai konvensi.

Method HTTP yang digunakan adalah GET, POST, PUT, dan DELETE. GET bersifat aman (safe) karena hanya membaca data tanpa mengubah state server. POST digunakan untuk menambahkan data baru dan status keberhasilannya adalah 201 Created. PUT menggantikan keseluruhan isi resource sehingga body request harus memuat seluruh field wajib, dan bersifat idempoten. DELETE menghapus resource dan juga bersifat idempoten.

Status code HTTP menunjukkan hasil pemrosesan request. Status 200 OK digunakan untuk GET, PUT, dan DELETE yang berhasil, 201 Created untuk POST yang berhasil, 400 Bad Request untuk request yang isinya tidak valid, dan 404 Not Found untuk data dengan id yang tidak ada maupun endpoint yang tidak terdaftar.

Pada praktikum ini data disimpan pada array di dalam memori (bukan database) sesuai ketentuan, dan aplikasi dikonfigurasi agar dapat berjalan di localhost maupun di-deploy pada platform serverless Vercel melalui `module.exports = app` dan berkas `vercel.json`.

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	v26.7.0 (LTS) - runtime server-side
2	Visual Studio Code	Editor teks untuk menulis app.js
3	npm	v11.19.0 - package manager
4	Express.js	v4.19.2 - framework RESTful API (dependency)
5	nodemon	v3.1.4 - auto-restart saat development (devDependency)
6	Thunder Client	Perangkat uji endpoint API
7	Git	v2.53.0 - version control
8	GitHub	Repository publik tugas1-restful-2428240094
9	Vercel	Platform deployment serverless

IV. Langkah Kerja

1. Inisialisasi project Express.js

Package.json dilengkapi entry point "main": "app.js" serta script "start": "node app.js" dan "dev": "nodemon app.js". Middleware express.json() dipasang agar request dengan Content-Type: application/json dapat dibaca melalui req.body. npm init -y npm install express npm install --save-dev nodemon

```
{ } package.json > ...
1  {
2    "name": "tugas1-restful-2428240094",
3    "version": "1.0.0",
4    "description": "Tugas 1 - RESTful API Murni dengan Express.js (Topik 26: Smartphone, SISB)",
5    "main": "app.js",
6    "scripts": {
7      "start": "node app.js",
8      "dev": "nodemon app.js"
9    },
10   "keywords": [
11     "express",
12     "restful-api",
13     "tugas1"
14   ],
15   "author": "2428240094",
16   "license": "MIT",
17   "dependencies": {
18     "express": "^4.19.2"
19   },
20   "devDependencies": {
21     "nodemon": "^3.1.4"
22   }
23 }
```

2. Menyiapkan data awal pada array di memori

Empat data awal disimpan pada array di dalam memori, bukan pada database. Penomoran id dibuat otomatis melalui variabel nextId sehingga id tidak perlu dikirim pada body request. Kelima field wajib topik 26 didaftarkan pada FIELD_WAJIB untuk dipakai oleh validasi POST dan PUT. const smartphones = [

```
{ id: 1, merek: "Xiaomi", model: "Redmi Note 14", ramGb: 8,
  penyimpananGb: 256, harga: 3299000 },
```

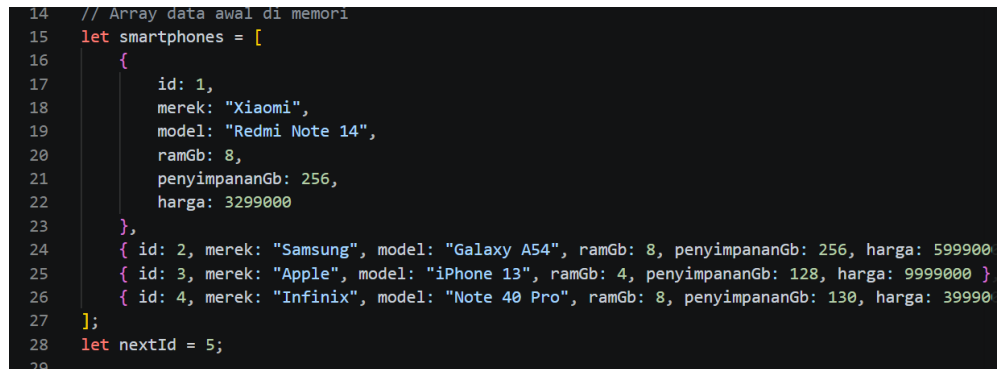
```
{ id: 2, merek: "Samsung", model: "Galaxy A54", ramGb: 8,
  penyimpananGb: 256, harga: 5999000 },
```

```
{ id: 3, merek: "Apple", model: "iPhone 13", ramGb: 4, penyimpananGb:
  128, harga: 9999000 },
```

```

    { id: 4, merek: "Infinix", model: "Note 40 Pro", ramGb: 8,
    penyimpananGb: 130, harga: 3999000 }
  ];
  let nextId = 5;

```



```

14 // Array data awal di memori
15 let smartphones = [
16   {
17     id: 1,
18     merek: "Xiaomi",
19     model: "Redmi Note 14",
20     ramGb: 8,
21     penyimpananGb: 256,
22     harga: 3299000
23   },
24   { id: 2, merek: "Samsung", model: "Galaxy A54", ramGb: 8, penyimpananGb: 256, harga: 5999000 },
25   { id: 3, merek: "Apple", model: "iPhone 13", ramGb: 4, penyimpananGb: 128, harga: 9999000 },
26   { id: 4, merek: "Infinix", model: "Note 40 Pro", ramGb: 8, penyimpananGb: 130, harga: 3999000 }
27 ];
28 let nextId = 5;
29

```

3. Membuat route GET, filter query string, dan route GET per id

Route parameter hanya dipakai untuk :id dan dibaca melalui req.params.id, sedangkan filter memakai query string yang dibaca melalui req.query. Kedua response GET ini mengembalikan data langsung tanpa pembungkus status dan message. Hasil filter tetap berupa array dan boleh kosong dengan status 200.

// GET /smartphones & Filter (GET /smartphones?merek=nilai)

```

app.get('/smartphones', (req, res) => {
  // Jika ada query string 'merek'
  if (req.query.merek) {
    const filteredData = smartphones.filter(
      (s) => s.merek.toLowerCase() === req.query.merek.toLowerCase()
    );
    return res.status(200).json(filteredData);
  }
  // Jika tidak ada filter, tampilkan semua
  res.status(200).json(smartphones);
});

```

```
// GET /smartphones/:id (Ambil satu data)

app.get('/smartphones/:id', (req, res) => {

  const id = parseInt(req.params.id);

  const data = smartphones.find((s) => s.id === id);

  if (!data) {

    return res.status(404).json({

      status: "error",

      message: `Data dengan id ${id} tidak ditemukan`,

      data: null

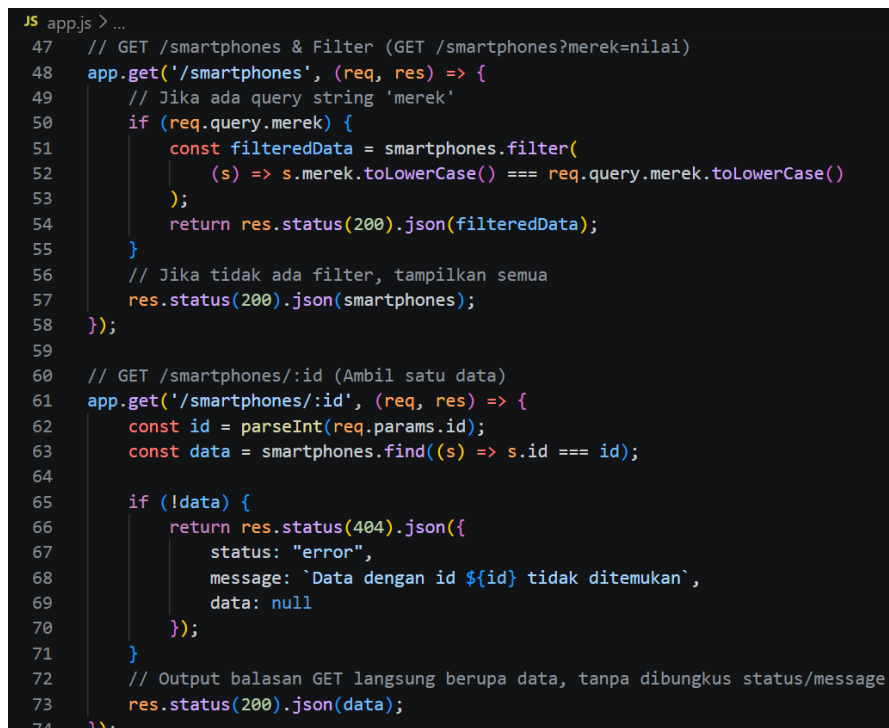
    });

  }

  // Output balasan GET langsung berupa data, tanpa dibungkus status/message

  res.status(200).json(data);

});
```



```
JS appjs > ...
47 // GET /smartphones & Filter (GET /smartphones?merek=nilai)
48 app.get('/smartphones', (req, res) => {
49   // Jika ada query string 'merek'
50   if (req.query.merek) {
51     const filteredData = smartphones.filter(
52       (s) => s.merek.toLowerCase() === req.query.merek.toLowerCase()
53     );
54     return res.status(200).json(filteredData);
55   }
56   // Jika tidak ada filter, tampilkan semua
57   res.status(200).json(smartphones);
58 });
59
60 // GET /smartphones/:id (Ambil satu data)
61 app.get('/smartphones/:id', (req, res) => {
62   const id = parseInt(req.params.id);
63   const data = smartphones.find((s) => s.id === id);
64
65   if (!data) {
66     return res.status(404).json({
67       status: "error",
68       message: `Data dengan id ${id} tidak ditemukan`,
69       data: null
70     });
71   }
72   // Output balasan GET langsung berupa data, tanpa dibungkus status/message
73   res.status(200).json(data);
74 }
```

4. Membuat route POST, PUT, dan DELETE beserta validasi

POST menambah data baru dengan status 201 dan id otomatis dari nextId++.

PUT melakukan penggantian penuh seluruh field wajib dengan id tetap, dan memeriksa keberadaan id lebih dahulu sehingga id yang tidak ada menghasilkan 404 sebelum validasi field. DELETE menghapus data dengan splice lalu mengirim status 200, pesan yang menyebut id, dan nilai data null. Helper fieldKosong() mengembalikan daftar nama field wajib yang kosong sehingga pesan error menyebutkan field yang bermasalah.

```
// POST /smartphones (Tambah data baru)
app.post('/smartphones', (req, res) => {
  const { merek, model, ramGb, penyimpananGb, harga } = req.body;

  // Validasi field wajib
  if (!merek || !model || !ramGb || !penyimpananGb || !harga) {
    return res.status(400).json({
      status: "error",
      message: "Field merek, model, ramGb, penyimpananGb, dan harga wajib
diisi",
      data: null
    });
  }

  const baru = { id: nextId++, merek, model, ramGb, penyimpananGb, harga };
  smartphones.push(baru);

  res.status(201).json({
    status: "success",
    message: "Data berhasil ditambahkan",
    data: baru
  });
});

// PUT /smartphones/:id (Ubah seluruh data)
app.put('/smartphones/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const { merek, model, ramGb, penyimpananGb, harga } = req.body;

  // Validasi field wajib
  if (!merek || !model || !ramGb || !penyimpananGb || !harga) {
```

```
        status: "error",
        message: "Field merek, model, ramGb, penyimpananGb, dan harga wajib
diisi",
        data: null
    });
}

const index = smartphones.findIndex((s) => s.id === id);
if (index === -1) {
    return res.status(404).json({
        status: "error",
        message: `Data dengan id ${id} tidak ditemukan`,
        data: null
    });
}

// Mengganti penuh data pada index tersebut
smartphones[index] = { id, merek, model, ramGb, penyimpananGb, harga };

res.status(200).json({
    status: "success",
    message: "Data berhasil diubah",
    data: smartphones[index]
});
});

// DELETE /smartphones/:id (Hapus data)
app.delete('/smartphones/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = smartphones.findIndex((s) => s.id === id);

    if (index === -1) {
        return res.status(404).json({
            status: "error",
            message: `Data dengan id ${id} tidak ditemukan`,
            data: null
        });
    }

    smartphones.splice(index, 1);

    res.status(200).json({
        status: "success",
```

message: `Data smartphone dengan id \${id} berhasil dihapus`,
data: null

```
});
});

76 // POST /smartphones (Tambah data baru)
77 app.post('/smartphones', (req, res) => {
78   const { merek, model, ramGb, penyimpananGb, harga } = req.body;
79
80   // Validasi field wajib
81   if (!merek || !model || !ramGb || !penyimpananGb || !harga) {
82     return res.status(400).json({
83       status: "error",
84       message: "Field merek, model, ramGb, penyimpananGb, dan harga wajib diisi",
85       data: null
86     });
87   }
88
89   const baru = { id: nextId++, merek, model, ramGb, penyimpananGb, harga };
90   smartphones.push(baru);
91
92   res.status(201).json({
93     status: "success",
94     message: "Data berhasil ditambahkan",
95     data: baru
96   });
97 });
98
99 // PUT /smartphones/:id (Ubah seluruh data)
100 app.put('/smartphones/:id', (req, res) => {
101   const id = parseInt(req.params.id);
102   const { merek, model, ramGb, penyimpananGb, harga } = req.body;
103   // Validasi field wajib
104   if (!merek || !model || !ramGb || !penyimpananGb || !harga) {
105     return res.status(400).json({
106       status: "error",
107       message: "Field merek, model, ramGb, penyimpananGb, dan harga wajib diisi",
108       data: null
109     });
110   }
111   const index = smartphones.findIndex((s) => s.id === id);
112   if (index === -1) {
113     return res.status(404).json({
114       status: "error",
115       message: `Data dengan id ${id} tidak ditemukan`,
116       data: null
117     });
118   }
119   // Mengganti penuh data pada index tersebut
120   smartphones[index] = { id, merek, model, ramGb, penyimpananGb, harga };
121   res.status(200).json({
122     status: "success",
123     message: "Data berhasil diubah",
124     data: smartphones[index]
125   });
126 });
```

```

132 // DELETE /smartphones/:id (Hapus data)
133 app.delete('/smartphones/:id', (req, res) => {
134     const id = parseInt(req.params.id);
135     const index = smartphones.findIndex((s) => s.id === id);
136
137     if (index === -1) {
138         return res.status(404).json({
139             status: "error",
140             message: `Data dengan id ${id} tidak ditemukan`,
141             data: null
142         });
143     }
144
145     smartphones.splice(index, 1);
146
147     res.status(200).json({
148         status: "success",
149         message: `Data smartphone dengan id ${id} berhasil dihapus`,
150         data: null
151     });
152 });

```

5. Menambahkan catch-all 404, konfigurasi Vercel, dan pengujian endpoint

Middleware catch-all diletakkan setelah seluruh route sehingga endpoint yang tidak terdaftar tetap dijawab dengan JSON 404, bukan HTML bawaan Express. `app.listen()` hanya dijalankan saat lokal, sedangkan `module.exports = app` digunakan agar aplikasi berjalan pada serverless Vercel. Verifikasi dilakukan menggunakan Thunder Client pada localhost maupun pada URL Vercel, termasuk skenario gagal 404 dan 400.

// Middleware catch-all 404 (Untuk route tidak dikenal)

```

app.use((req, res) => {
    res.status(404).json({
        status: "error",
        message: "Endpoint tidak ditemukan",
        data: null
    });
});

```

```
// Menjalankan server (Hanya lokal + ekspor untuk Vercel)

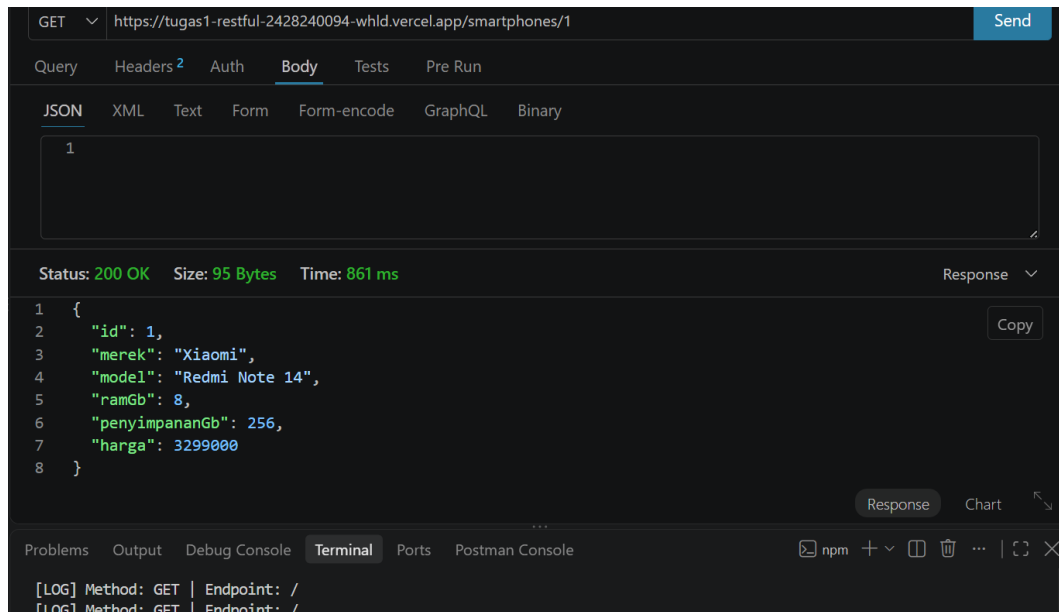
const PORT = process.env.PORT || 3000;

if (process.env.NODE_ENV !== 'production') {
    app.listen(PORT, () => console.log(`Server berjalan di
    http://localhost:${PORT}`));
}

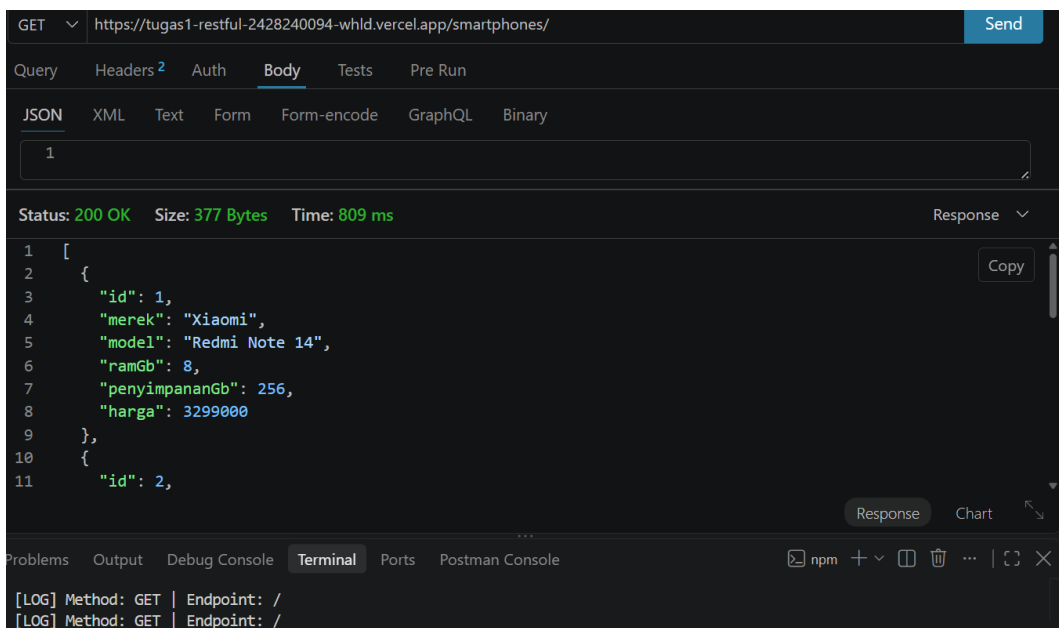
module.exports = app;
```

```
154 // Middleware catch-all 404 (Untuk route tidak dikenal)
155 app.use((req, res) => {
156     res.status(404).json({
157         status: "error",
158         message: "Endpoint tidak ditemukan",
159         data: null
160     });
161 });
162
163 // Menjalankan server (Hanya lokal + ekspor untuk Vercel)
164 const PORT = process.env.PORT || 3000;
165 if (process.env.NODE_ENV !== 'production') {
166     app.listen(PORT, () => console.log(`Server berjalan di http://localhost:${PORT}`));
167 }
168
169 module.exports = app;
```

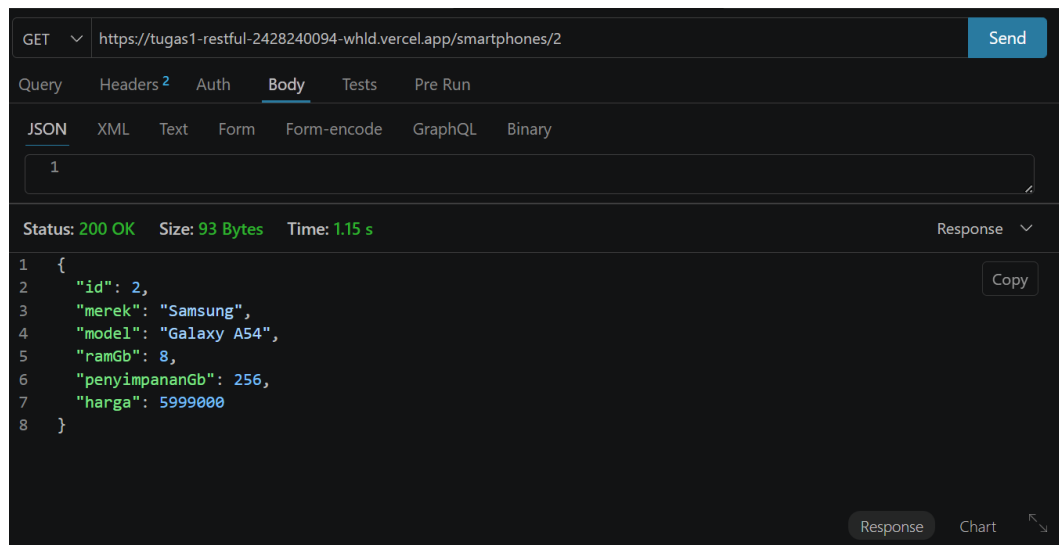
V. Hasil dan Pembahasan



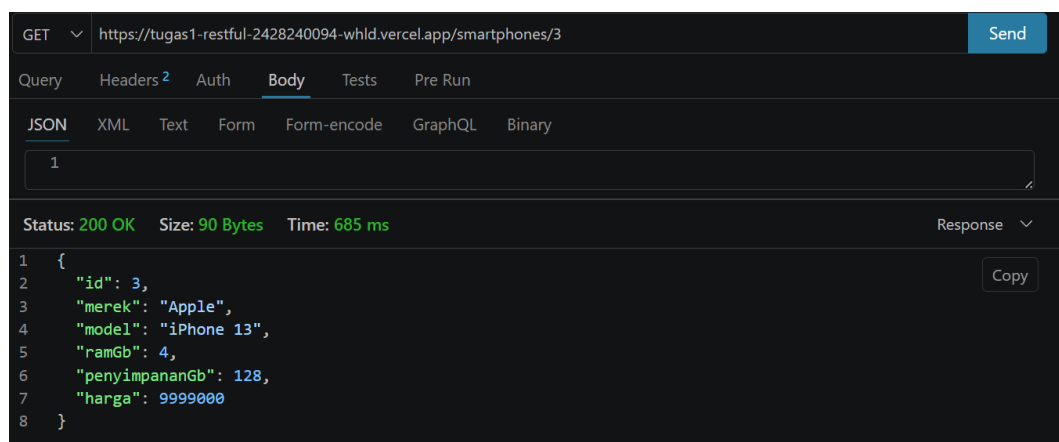
Gambar 1. Hasil pengujian endpoint `GET /smartphones/1` pada URL Vercel



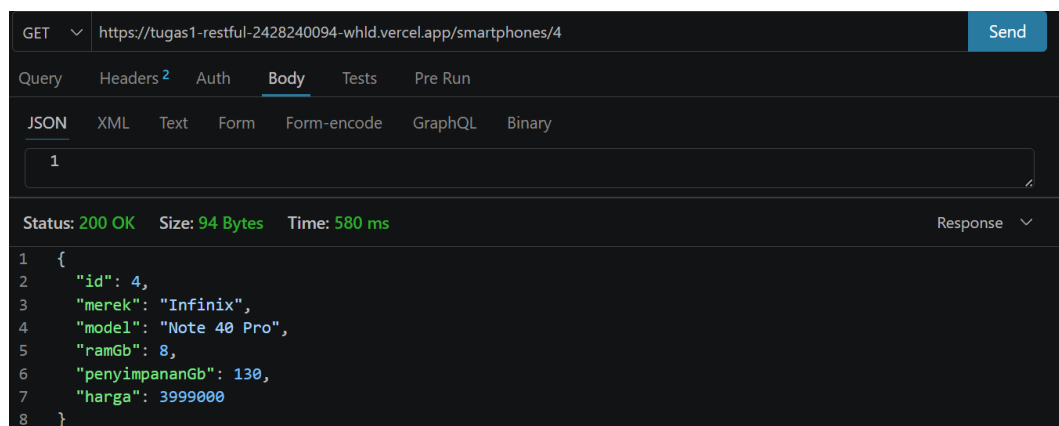
Gambar 2. Hasil pengujian endpoint `GET /smartphones/` pada URL Vercel



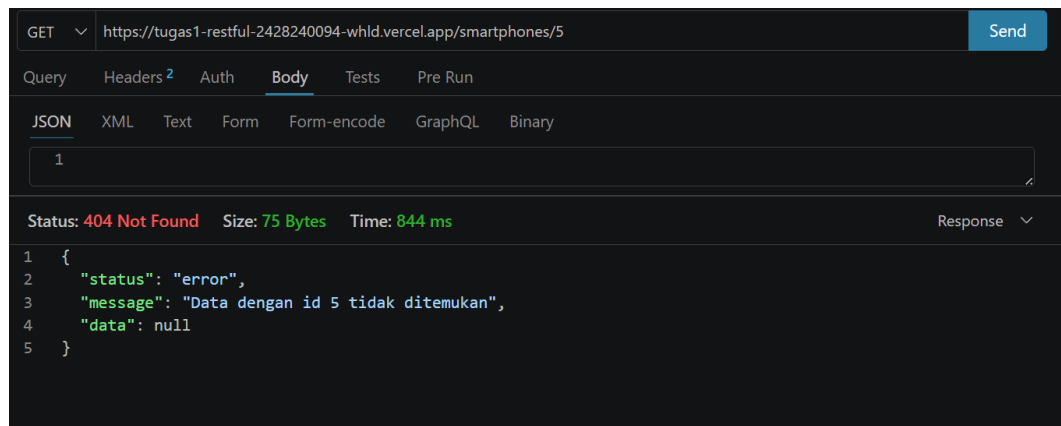
Gambar 3. Hasil pengujian endpoint GET /smartphones/2 pada URL Vercel



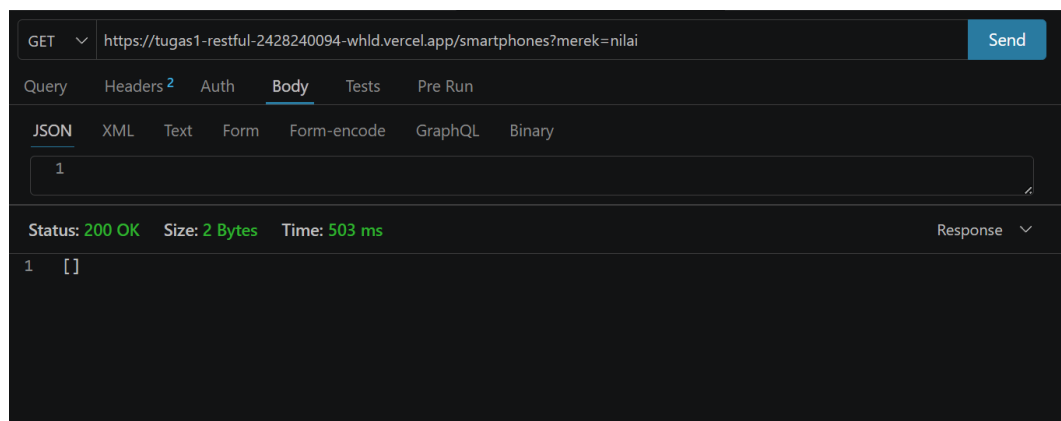
Gambar 4. Hasil pengujian endpoint GET /smartphones/3 pada URL Vercel



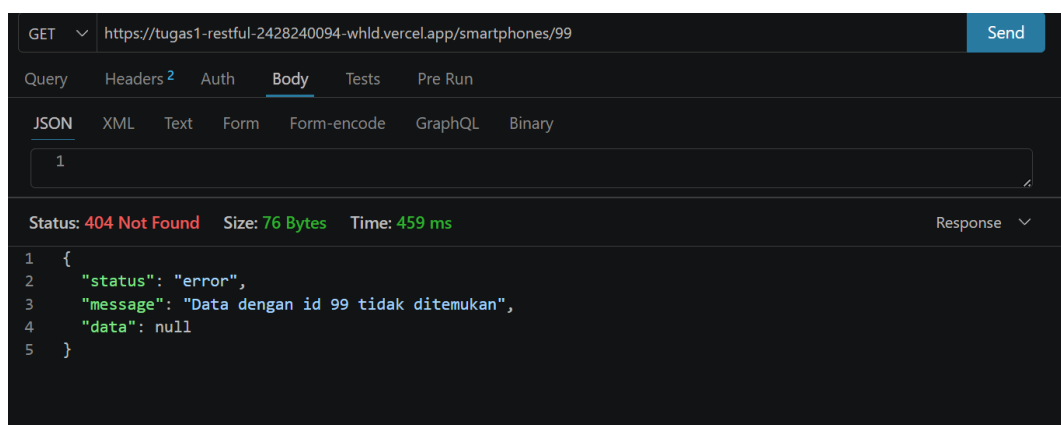
Gambar 5. Hasil pengujian endpoint GET /smartphones/4 pada URL Vercel



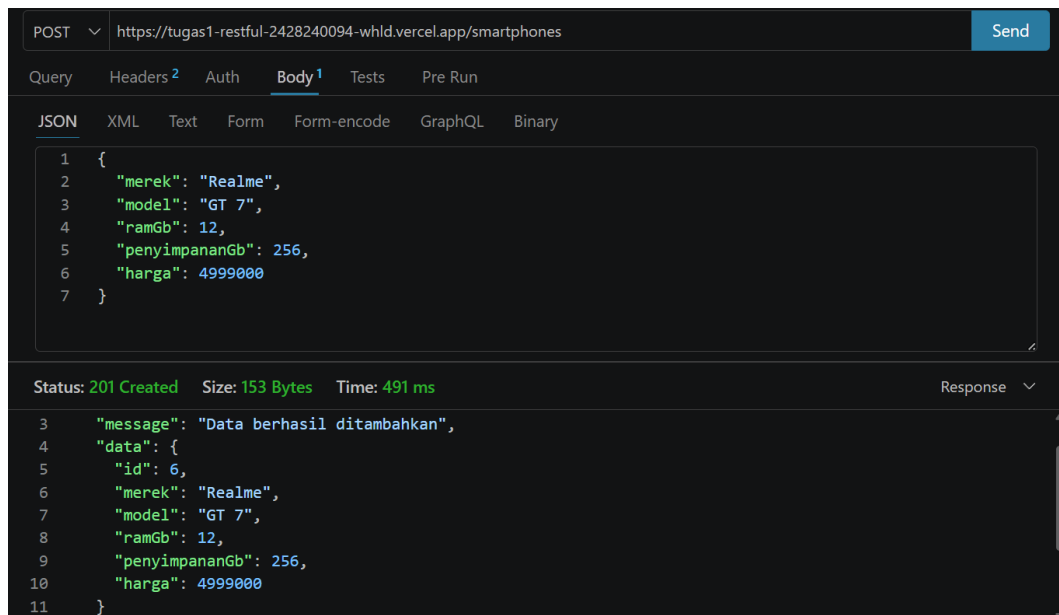
Gambar 6. Hasil pengujian endpoint GET /smartphones/5 pada URL Vercel



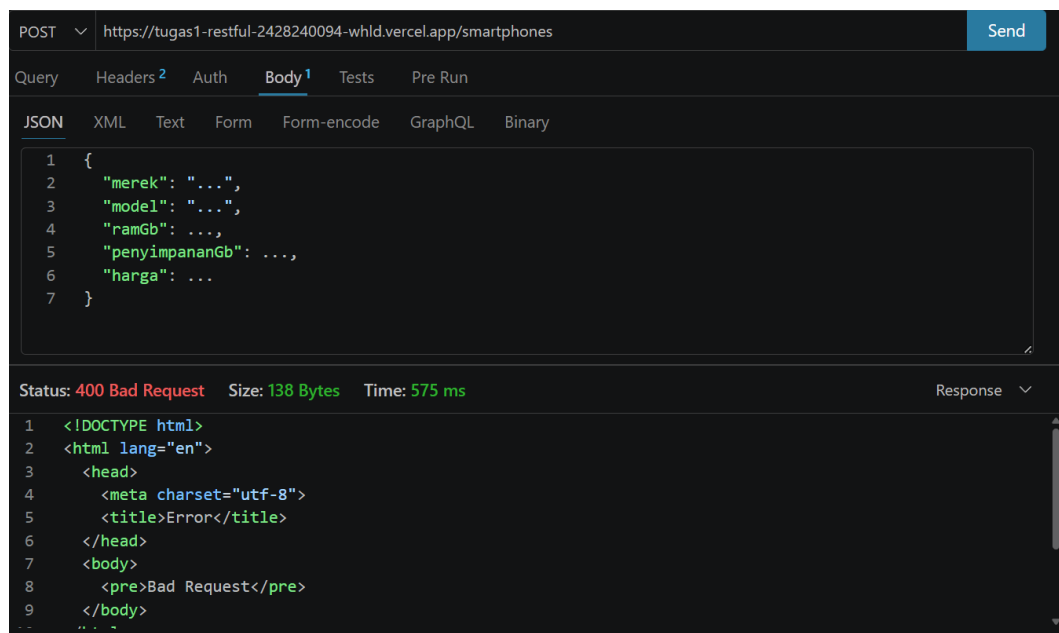
Gambar 7. Hasil pengujian endpoint GET /smartphones?merek=nilai pada URL Vercel



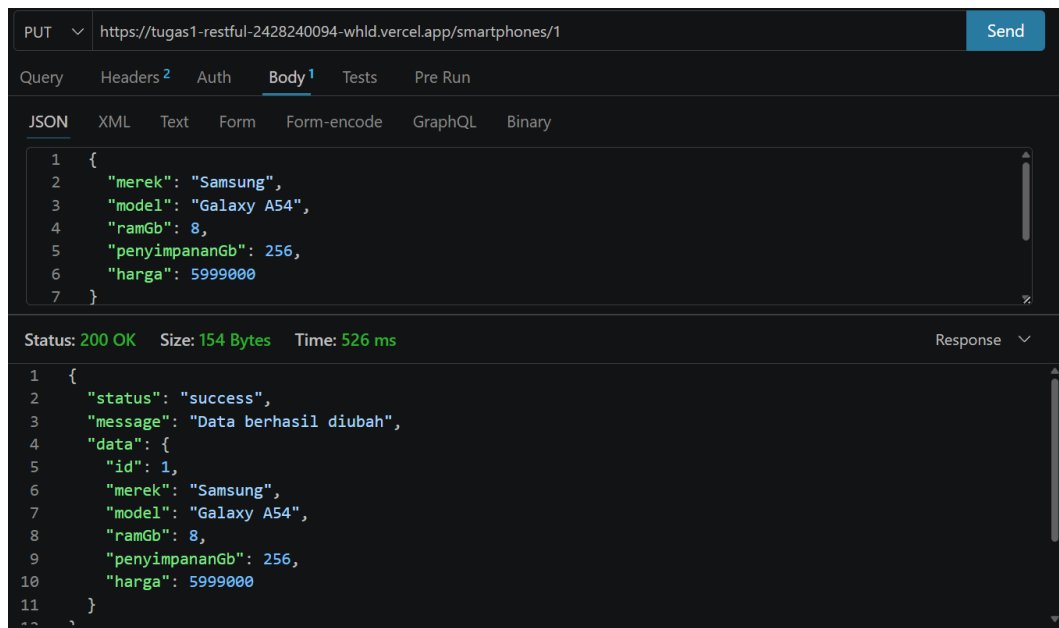
Gambar 8. Hasil pengujian endpoint GET /smartphones/99 pada URL Vercel



Gambar 9. Hasil pengujian endpoint `POST /smartphones` (body lengkap) pada URL Vercel



Gambar 10. Hasil pengujian endpoint `POST /smartphones` (field wajib kosong) pada URL Vercel



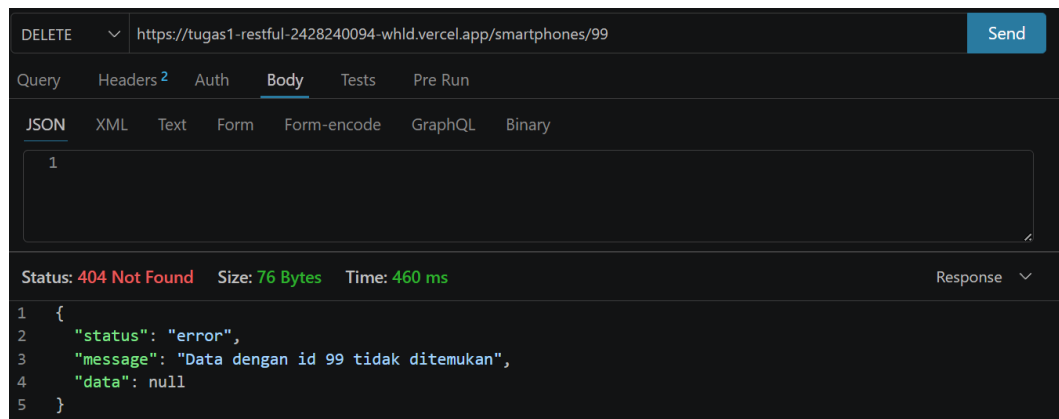
Gambar 11. Hasil pengujian endpoint PUT /smartphones/1 (body lengkap) pada URL Vercel



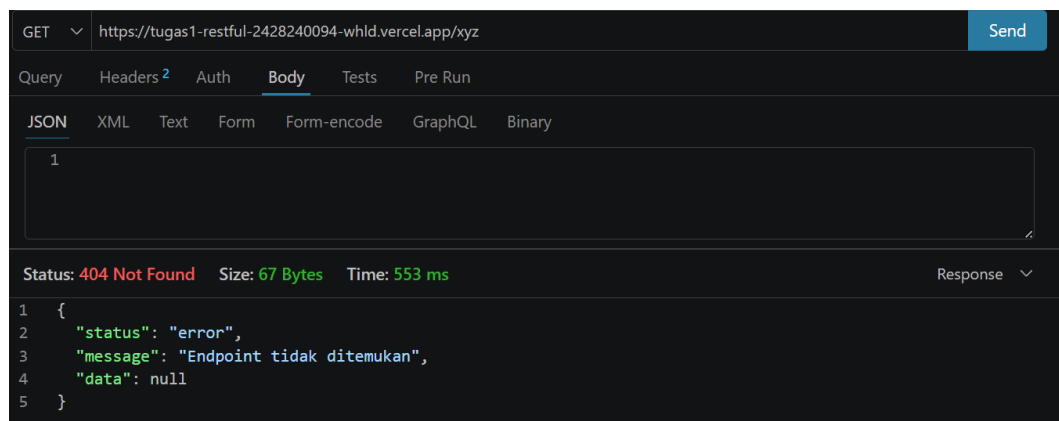
Gambar 12. Hasil pengujian endpoint PUT /smartphones/99 (body lengkap) pada URL Vercel



Gambar 13. Hasil pengujian endpoint DELETE /smartphones/1 pada URL Vercel



Gambar 14. Hasil pengujian endpoint DELETE /smartphones/99 pada URL Vercel



Gambar 15. Hasil pengujian endpoint GET /xyz pada URL Vercel

No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
1	GET	/smartphones	—	200	200	Sesuai
2	GET	/smartphones/1	—	200	200	Sesuai
3	GET	/smartphones/99	—	404	404	Sesuai
4	GET	/smartphones?merek=nilai	—	200	200	Sesuai
5	POST	/smartphones	Body lengkap	201	201	Sesuai
6	POST	/smartphones	Field wajib kosong	400	400	Sesuai
7	PUT	/smartphones/1	Body lengkap	200	200	Sesuai
8	PUT	/smartphones/99	Body lengkap	404	404	Sesuai
9	DELETE	/smartphones/1	—	200	200	Sesuai
10	DELETE	/smartphones/99	—	404	404	Sesuai
11	GET	/xyz (catch all)	—	404	404	Sesuai

Tabel 1. Hasil pengujian endpoint pada URL Vercel

Pembahasan:

Seluruh endpoint menghasilkan response sesuai ketentuan. Response GET mengembalikan data langsung tanpa pembungkus status dan message, yaitu objek untuk /smartphones/:id dan array untuk /smartphones maupun hasil filter. Sebaliknya response POST, PUT, DELETE, dan seluruh error memakai struktur { status, message, data } sesuai format yang diminta.

Pemilihan status code mengikuti semantik HTTP. Status 201 dipakai pada POST karena sumber daya baru berhasil diciptakan, dan status 200 dipakai pada GET, PUT, serta DELETE yang berhasil. Status 400 diberikan ketika field wajib kosong, sedangkan status 404 diberikan ketika data dengan id yang diminta tidak ada atau endpoint tidak terdaftar. Pada route PUT, pemeriksaan id dilakukan sebelum validasi field sehingga id yang tidak ada selalu menghasilkan 404.

PUT diterapkan sebagai penggantian penuh, bukan penggabungan dengan data lama, sehingga seluruh field wajib harus dikirim ulang dan id tetap sama seperti sebelumnya. Helper `keAngka()` menormalkan nilai angka yang terkirim sebagai string menjadi number, sehingga data pada array konsisten bertipe dan dapat difilter dengan benar. Validasi semua field wajib dilakukan bersamaan melalui helper `fieldKosong()` sehingga pesan error dapat menyebutkan field mana saja yang kosong.

Perlu diperhatikan bahwa data pada array di memori dapat kembali ke data awal setelah beberapa waktu ketika aplikasi berjalan pada Vercel karena sifat serverless yang dapat mengulang instance tanpa menyimpan state. Hal ini wajar dan tidak mengganggu karena setiap request tetap menghasilkan response yang benar. Selain itu, urutan pengujian memengaruhi status yang tercatat: apabila suatu id telah dihapus pada skenario DELETE, maka pengujian PUT pada id yang sama akan menghasilkan 404, bukan 400. Karena itu skenario gagal diuji memakai id yang belum terpengaruh.

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	Data pada array di memori dapat kembali ke data awal setelah beberapa waktu ketika aplikasi berjalan di Vercel.	Memahami bahwa array di memori memang diperbolehkan oleh soal, dan setiap request tetap menghasilkan response yang benar.
2	Body request dalam format JSON yang rusak membuat Express mengirim response HTML bawaan.	Menambahkan error handler setelah catch-all middleware sehingga response yang bukan JSON tetap diubah menjadi JSON.
3	Filter dengan nilai yang tidak tersedia pada data, misalnya ?merek=Nokia, mengembalikan array kosong.	Sesuai ketentuan soal hasil filter boleh kosong dan tetap memakai status 200, sehingga tidak diperlakukan sebagai error.
4	Kode yang hanya mengganti nama resource dari mahasiswa lain akan bernilai 0.	Seluruh route, nama field, validasi, dan pesan error ditulis khusus untuk topik smartphone sehingga kode tidak identik dengan mahasiswa lain.

VII. Kesimpulan

RESTful API murni dengan Express.js berhasil dibuat untuk Topik 26, yaitu Toko Gadget:

Smartphone, dengan enam endpoint yang sesuai konvensi REST: GET /smartphones, GET /smartphones/:id, POST /smartphones, PUT /smartphones/:id, DELETE /smartphones/:id, dan filter GET /smartphones?merek=.

Seluruh field wajib topik 26, yaitu merek, model, ramGb, penyimpananGb, dan harga, berhasil divalidasi. Request dengan field wajib kosong menghasilkan status 400, sedangkan request dengan id yang tidak ada maupun endpoint yang tidak terdaftar menghasilkan status 404.

Response API berjalan sesuai format yang diminta: GET mengembalikan data langsung, sedangkan POST, PUT, DELETE, dan error memakai struktur { status, message, data } dengan status code 200, 201, 400, dan 404 yang tepat, serta tidak ditemukan adanya `res.send()` berisi teks atau HTML.

Dengan demikian ketiga tujuan pada bagian I telah tercapai: project RESTful berbasis

Express.js berhasil dibangun dengan routing, penggunaan method HTTP, dan response JSON murni yang sesuai ketentuan, serta telah diuji pada localhost maupun URL Vercel.

Daftar Pustaka

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. UC Irvine eScholarship. <https://escholarship.org/uc/item/9cb5261d>

Fielding, R., Nottingham, M., & Reschke, J. (2022). *HTTP semantics* (RFC No. 9110). Internet Engineering Task Force. <https://doi.org/10.17487/RFC9110>

Masse, M. (2011). *REST API design rulebook: Designing consistent RESTful web service interfaces*. O'Reilly Media.

Node.js. (n.d.). *Modules: CommonJS modules*. Node.js Documentation. Retrieved October 5, 2026, from <https://nodejs.org/api/modules.html>

Vercel. (n.d.-a). *Project configuration (vercel.json)*. Vercel Documentation. Retrieved October 5, 2026, from <https://vercel.com/docs/projects/project-configuration>

Vercel. (n.d.-b). *Using Express.js with Vercel*. Vercel Guides. Retrieved October 5, 2026, from <https://vercel.com/guides/using-express-with-vercel>

Lampiran

Tautan Repository GitHub:

<https://github.com/fellisaanastasya01/tugas1-restful-2428240094>

Tautan Deployment (jika ada):

<https://tugas1-restful-2428240094-whld.vercel.app/>

Riwayat Commit:

 fellisaanastasya01 update README.md ✓		3565f2e · 2 hours ago	 23 Commits
 .gitignore	init project express: middleware json, data awal smartph...	yesterday	
 Laporan_Tugas1_2428240094_FellisaAnasta...	Add files via upload	5 hours ago	
 README.md	update README.md	2 hours ago	
 app.js	Penambahan 1 produk	8 hours ago	
 package-lock.json	init project express: middleware json, data awal smartph...	yesterday	
 package.json	init project express: middleware json, data awal smartph...	yesterday	
 vercel.json	init project express: middleware json, data awal smartph...	yesterday	